FrontierLab Mini Program Project in theoretical Physics

SED Computing Pipeline for Simulated Galaxies

Technical Documentation

Author:	Lennart Rathjen
Last updated:	May 29, 2026
Supervisor:	Prof. Kentaro Nagamine, Ph-D.[1]
Compatible simulation format:	GADGET-4 Osaka HDF5
Main output format:	Python `.npy` galaxy dictionaries

[1] Theoretical Astrophysics, Department of Earth and Space Science, Graduate School of Science, The University of Osaka, Toyonaka, Japan

This project was carried out within the academic exchange framework between the University of Bremen and The University of Osaka.

Abstract

This document outlines the architecture, implementation, scientific assumptions, and algorithmic workflow of a post-processing pipeline designed to compute spectral energy distributions (SEDs) and broadband photometry for simulated galaxies. Optimized for *Osaka GADGET-4* cosmological simulation outputs, the pipeline extracts stellar components from subhalos, serializes reusable galaxy datasets, and determines physical lookback ages. Utilizing a precomputed master grid from the Flexible Stellar Population Synthesis (`FSPS`) library, the software executes memory-optimized, bilinear log-flux interpolations to derive both integrated galaxy SEDs and resolved per-particle broadband magnitudes.

This documentation serves as a comprehensive technical manual and reference guide for future students, collaborators, and researchers continuing this project or implementing spectral synthesis frameworks within their own cosmological analysis workflows.

Contents

1 Introduction

Modern cosmological simulations provide detailed information about the formation and evolution of galaxies, including the masses, positions, formation times, and metallicity of individual stellar particles. These simulations are highly successful in modeling the large-scale structure of the Universe and the assembly histories of galaxies. However, the raw simulation output does not directly correspond to observable quantities measured by astronomical survey.

Observational studies typically probe galaxies through their emitted light, such as spectral energy distributions (SEDs), colors, luminosities, and broadband magnitudes. In order to compare simulated galaxies with real observations, the intrinsic stellar populations produced by the simulation must therefore be transformed into synthesis observables using stellar population synthesis models.
The purpose of this pipeline is to ridge this gap by combining GADGET-4 simulation outputs with precomputed stellar population synthesis grids generated with FSPS (Flexible Stellar Population Synthesis). The pipeline extracts stellar populations from simulates synthetic spectra for individual star particles, and computed integrated galaxy SEDs and broadband photometric quantities.

The resulting data produces provide a physically motivated connection between numerical galaxy formation simulations and observable galaxy properties, enabling direct comparison with observational surveys and supporting further scientific analysis and visualization.

The structure of this documentation is as follows. Section 2 summarizes the recommended software environment, requires Python packages, and notes regarding the installation of FSPS. Section 3 describes the expected input data structure, and the format of the generated output products.

Section 4 provides a high-level overview o the pipeline architecture, including snapshot processing, SED computing, and photometric synthesis. The subsequent sections (Sections 5-8) describe these pipeline stages in greater technical detail, including the underlying algorithms, implementation details, and relevant functions.

Section 8 additionally summarizes the available analysis and visualization utilities. Section 9 discusses computational performance, memory usage, and scaling behavior. Section 10 outlines known limitations and implementation assumptions, while Section 11 provides a complete reference of all major pipeline functions.

Important Note Please note, that provided Jupyter Notebook code `SEDComputing_MainPipeline.v` is entirely commented. Therefore, only the most important code snippets are mentioned here. For more detail please refer to the function in the respective code.

1.1 Scope of the Pipeline

This pipeline is designed for the post-processing of cosmological GADGET-4 simulations, with the goal of generating synthetics observable properties for simulated galaxies.
The pipeline supports and is designed for:

- read GADGET-4 snapshot and group catalogue output
- identify and extract stellar particle data from subhalos
- stores galaxies as reusable Python dictionary files,
- cosmological age and redshift conversion from formation scale factors,
- interpolating stellar population synthesis models,
- pre-computing FSPS spectral grids,
- computing integrated spectral energy distributions (SEDs),
- and deriving broadband photometric quantities.

The current implementation does not include dust attenuation, nebular emission, radiative emission, or observational noise models. However, the pipeline is designed in a modular way and can therefore be extended or adapted to different scientific applications, stellar synthesis models, and filter systems.

2 Requirements and Installation

The pipeline is primarily designed for Linux-based systems and can be executed on local workstations, or university HPC systems with shared filesystems. However, please note, that the shared code mainly is for Jupyter Notebooks (.ipynb) and adapted for Python styled notebooks (.py). A Conda environment is recommended in order to isolate dependencies and simplify package management. The pipeline is compatible with Python 3.8 and newer, while the recommended and tested versions are Python 3.10 and Python 3.11.

2.1 Required Python Packages

Table 1 lists the main Python packages required for running the pipeline.

Table 1: Required Python packages.

Package	**Purpose**	**Installation**
`numpy`	Numerical arrays and file I/O	`pip install numpy`
`h5py`	Reading HDF5 simulation files	`pip install h5py`
`matplotlib`	Plotting and visualisation	`pip install matplotlib`
astropy	Cosmology and unit handling	`pip install astropy`
`tqdm`	Progress bars	`pip install tqdm`
fsps / **python-fsps**	FSPS interface and filters	see below
`os`	File-system operations	built-in
`glob`	File pattern matching	built-in
`re`	Regular expressions	built-in
`shutil`	File backup operations	built-in

2.2 Example Conda Environment

An example Conda environment can be created via:

```
conda create -n env_name python=3.11
conda activate env_name

pip install numpy h5py matplotlib astropy tqdm fsps
```

Alternatively, `python-fsps` can often be installed more reliably via conda-forge:

```
conda install -c conda-forge python-fsps
```

2.3 FSPS Installation Notes

The `python-fsps` package requires both the FSPS source code and a working Fortran compiler. Depending on the local system configuration, installation via `pip` may fail due to compiler or library issues. In such a case, installation through `conda-forge` is usually more robust, while a `git-installation` will be the most robust and easiest option.

Please refer to the official documentations for a more detailed description:

https://github.com/cconroy20/fsps/blob/master/README.md
https://python-fsps.readthedocs.io

2.4 HPC Considerations

For large simulations, the dominant computational bottlenecks are typically related to HDF5 I/O, spectral interpolating, and storage of large spectral datasets. For HPC environments, the following recommendations are generally useful:

- use local scratch storage whenever available,
- avoid excessive simultaneous I/O operations on shared filesystems,
- use chunked or `laptop` SED mode for very large galaxies,
- process snapshots independently when possible.

Depending on the wavelength resolution and the number of star particles, memory usage can become substantial when computing fully vectorized SEDs.

3 Data Structure

This section describes the expected directory layout, simulation input structure, intermediate galaxy storage, and the organization of the derived SED and photometric products generated by the pipeline.

3.1 Project Directory Structure

The pipeline assumes a predefined directory structure for simulation data, FSPS grids, intermediate galaxy storage, and notebook execution. The needed project layout is shown below:

```
Synthesizer_Osaka/
    BigData/
        FSPS_Grids/
        snapdir_xxx/
        groups_xxx/

    DataSave/
        Galaxies.Storage/
            ...

    GS.Main/
        JP.Notebooks/
            SEDComputing_MainPipeline.v1.ipynb

    PY.scr/
```

The pipeline internally relies on relative or absolute paths pointing to these directories. If alternative directory layouts are used, the corresponding paths within the notebook and helper scripts must be adapted accordingly.

3.2 Simulation Input

The current implementation assumes GADGET-4 HDF5 snapshot output together with SUBFIND group catalogues. Snapshot particle data and halo catalogues are expected to follow the standard GADGET-4 directory structure:

```
BigData/
    snapdir_020/
        snapshot_020.0.hdf5
        snapshot_020.1.hdf5
        ...

    groups_020/
        fof_subhalo_tab_020.0.hdf5
        fof_subhalo_tab_020.1.hdf5
        ...
```

The snapshot files contain particle data, while the group catalogs store FoF and subhalo information generated by SUBFIND.

The pipeline currently assumes:

- stellar particles are stored as `PartType4`,
- dark matter particles are stored as `PartType1`.

Additional details regarding the snapshot format and particle storage can be found in the official GADGET-4 documentation:

https://wwwmpa.mpa-garching.mpg.de/gadget4/01_index/

3.3 Galaxy Storage

After snapshot processing, each extracted galaxy is stored as an individual `.npy` file containing a Python dictionary with the associated particle data and derived quantities.

A typical storage hierarchy is:

```
DataSave/
    Galaxies.Storage/
        snapshot_020/
            Galaxy0072072_020.000000.npy
            Galaxy0066342_020.000001.npy
            ...
```

Galaxy files follow the naming convention:

```
Galaxy{N_star:07d}_{snapnum:03d}.{rank:06d}.npy
```

where:

- `N_star` denotes the number of stellar particles,
- `snapnum` is the snapshot number,
- `rank` corresponds to the galaxy rank after sorting by stellar particle number.

This naming scheme simplifies size-based sorting and parallelized processing of galaxy samples.

A typical galaxy dictionary contains entries such as:

```
gal.keys()

dict_keys([
    "mass",
    "pos",
    "age",
    "Z",
    "subhalo_id",
    ...
    "stellar_age",
    "z_snap",
    "z_form",
    "sed"
])
```

The galaxy dictionaries generated by the pipeline contain both the original particle properties extracted from the simulation snapshots and additional derived quantities computed during post-processing. Table 2 summarizes the most important entries commonly stored for each galaxy, including their description and units.

Table 2: Typical galaxy dictionary entries.

Key	Description	Units
`mass`	Stellar particle masses	$M_\odot$
`pos`	Particle positions	kpc
`age`	Stellar formation scale factor	dimensionless
`stellar_age`	Stellar population age	Gyr
`Z`	Stellar metallicity	absolute metallicity
`subhalo_id`	SUBFIND subhalo identifier	integer
`z_snap`	Snapshot redshift	dimensionless
`z_form`	Stellar formation redshift	dimensionless
`sed`	Stored SED products	dictionary

3.4 SED and Photometric Products

Computed spectral energy distributions are stored directly within the galaxy dictionary under:

```
gal["sed"][model_name][variant_name]
```

A typical SED entry has the form:

```
gal["sed"]["model_name"]["variant_name"] = {
    "wavelength":        lam,
    "sed":               sed,
    "grid_key":          variant_name,
    "imf":               getattr(grid, "imf", "unknown"),
```

```
    "photometry":        ...,
    "photometry_meta":   ...
}
```

Each SED entry therefore stores:

- the wavelength grid,
- the integrated spectral energy distribution,
- the associated FSPS grid identifier,
- the adopted IMF label,
- and optional photometric products.

For example:

```
gal["sed"]["FSPS"]["fsps_parsec_basel_chabrier"]
```

Photometric quantities are stored within the corresponding SED entry:

```
gal["sed"][model_name][variant_name]["photometry"]
```

The current implementation computes SDSS absolute magnitudes (`u`, `g`, `r`, `i`, `z`) for each stellar particle.

The default `model_name` used by the included FSPS grid creation pipeline is `FSPS`. Alternative stellar population synthesis grids may also be used, provided that the required dictionary structure and metadata keys are preserved.

3.5 Pipeline Data Flow

Figure 1 summarizes the overall data flow of the pipeline, beginning with raw GADGET-4 snapshots and ending with derived SEDs, photometric quantities, and scientific analysis products.

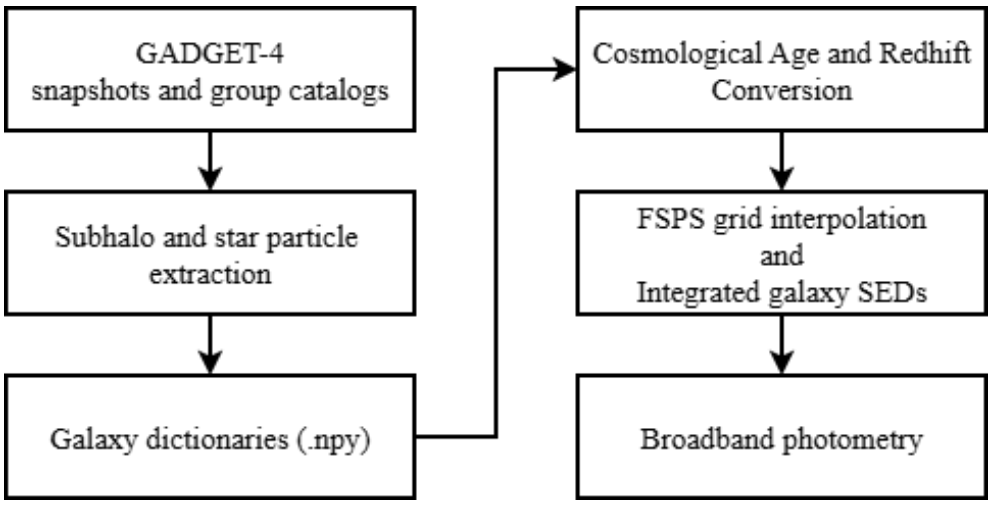

Figure 1: General data flow of the SED computing pipeline.

4 Pipeline Architecture

The pipeline is organized into four main processing stages, beginning with raw GADGET-4 simulation outputs and ending with derived spectral and photometric galaxy properties. Each stage additionally includes basic analysis and visualization utilities for validating the generated data products.

4.1 Stage 1: Snapshot Processing

This stage reads GADGET-4 snapshot and group catalog files, identifies available snapshots, and extracts stellar particle data for individual subhalos. Basic stellar and halo properties are computed and stored for each galaxy as an individual `.npy` dictionary.

The stage additionally provides several diagnostic and validation tools, including:

- halo mass distributions,
- stellar particle count distributions,
- subhalo position maps,
- stellar-to-halo mass relations,
- and resolution-limit diagnostics.

4.2 Stage 2: FSPS Grid Generation

This stage generates the precomputed stellar population synthesis grids required for efficient SED computation. The current implementation uses FSPS-based spectral grids covering stellar age and metallicity space.

Since the pipeline operates on precomputed grids, alternative stellar population synthesis models or parameter configurations can be included with only minor modifications to the grid-generation stage.

4.3 Stage 3: SED Computation

This stage forms the scientific core of the pipeline. Stellar ages and formation redshifts are derived from the simulation scale factors, after which precomputed FSPS spectra are interpolated for each stellar particle according to its age and metallicity.

The interpolated spectra are subsequently mass-weighted and summed to produce integrated galaxy spectral energy distributions.

4.4 Stage 4: Photometric Synthesis

This stage convolves the integrated spectra with SDSS filter transmission curves in order to derive broadband absolute AB magnitudes.

The current implementation produces intrinsic rest-frame photometry and does not include dust attenuation, radiative transfer effects, cosmological redshifting, or observational noise models.

5 Snapshot Processing

5.1 Overview

The snapshot-processing stage converts raw GADGET-4 simulation output into reusable galaxy catalogs. This preprocessing step is executed once before running the SED and photometric synthesis stages.

The pipeline automatically identifies available, loads the corresponding snapshot and SUBFIND group catalog files, extracts stellar particle data for each subhalo, and stores the resulting galaxy properties as individual `.npy` dictionaries. The main snapshot-processing workflow is handled by functions such as:

- `detect_all_snapnums(...)`,
- `load_all_XXX_for_snapnum(...))`,
- `merge_group_subalos(...)`,
- `extract_star_particles_global(...)`,
- `process_galaxies_in_snapshots(...)`.

5.2 Main Operations

Figure 2 summarizes the overall workflow of the snapshot processing, beginning with detecting all snapshots and ending by providing ready to use galaxies. The resulting galaxy catalogs form the basis for all later SED and photometric calculations.

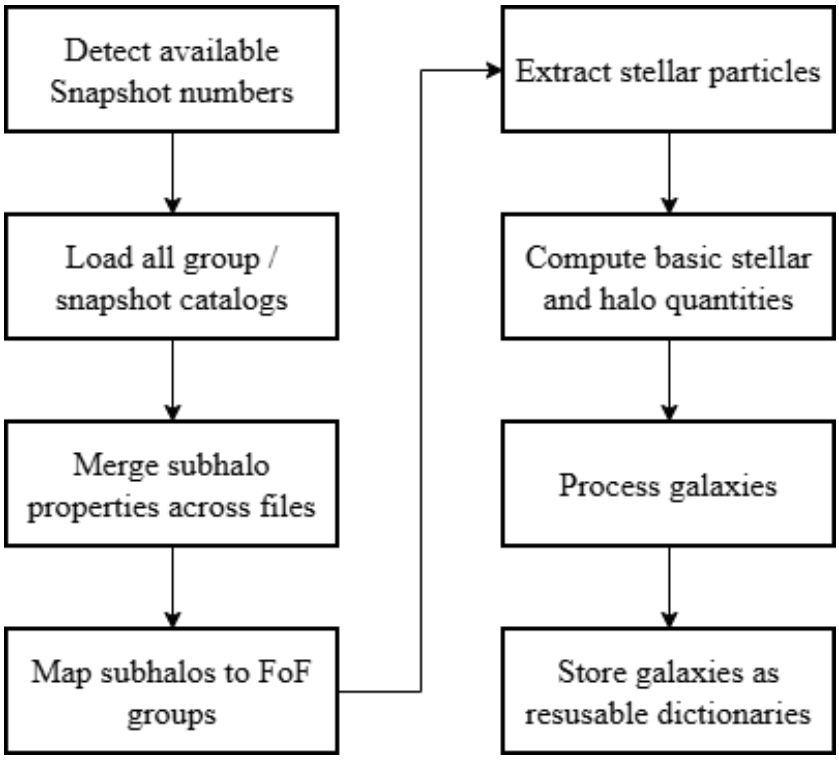

Figure 2: General snapshot processing workflow.

Subhalo Extraction The subhalos are extracted after merging the subhalo catalog with the groups catalog in the `process_galaxies_in_snapshot` function. The pipeline applies two selection criteria: the first selects all subhalos with $N_{\text{star}} \geq 0$, while the second enforces a physically motivated minimum number of star particles derived from the Stellar-to-Halo Mass Ratio (SHMR), as implemented below:

```
    # 1. Define minimum halo mass resolution (32 DM particles)
    m_dm = np.median(snap_files["PartType1"]["Masses"][:] *
       MASS_UNIT)
    M_halo_min = 32 * m_dm

    # 2. Compute local SHMR within a mass shell around M_halo_min
    ratio = M_star / M_halo
    mask_shell = (M_halo > M_halo_min/2) & (M_halo < 2*M_halo_min)
       & (M_star > 0)
    R_local = np.median(ratio[mask_shell])

    # 3. Calculate minimum required star particles
    M_star_min = R_local * M_halo_min
    m_star = np.median(snap_files["PartType4"]["Masses"][:] *
       MASS_UNIT)
    N_star_min = M_star_min / m_star

    # 4. Apply selection mask
    # mask = (N_star >= 0)            # Option 1
    mask = (N_star >= N_star_min)     # Option 2
    galaxy_ids = np.where(mask)[0]
```

5.3 Unit Conversions

The code uses the simulation Hubble parameter h to convert GADGET mass units into solar masses:

$$M_{\text{unit}} = \frac{10^{10}}{h}.$$

This converts masses stored approximately in units of $10^{10} M_{\odot}/h$ into physical solar masses. Additional conversions, such as cosmological age calculations and distance-related quantities, are performed using `astropy.cosmology`.

5.4 Computational Notes and Limitations

Snapshot processing is primarily limited by HDF5 I/O performance. Reading large numbers of snapshot chunks from shared storage can dominate the total runtime. The current implementation additionally assumes, that all stellar particles belonging to a subhalo are located within the same snapshot chunk file. This assumption is used by:

```
extract_star_particles_global(...)
```

Extremely large subhalos whose stellar particles span multiple snapshot chunks may therefore be extracted incompletely unless the extraction logic is extended to handle chunk

boundaries explicitly. The processing cost scales approximately linearly with the number of subhalos and stellar particles.

5.5 Diagnostic Visualization

The diagnostics summarize both subhalo and particle statistics. Furthermore, they visualize the total number of subhalos containing star particles compared to the entire subhalo population. Finally, a Stellar-to-Halo Mass Ratio (SHMR) plot is provided, which includes the *Behroozi2013* relation. This plot serves to evaluate whether the manual or the physically motivated selection criterion for subhalos should be applied, as illustrated in Figure 3.

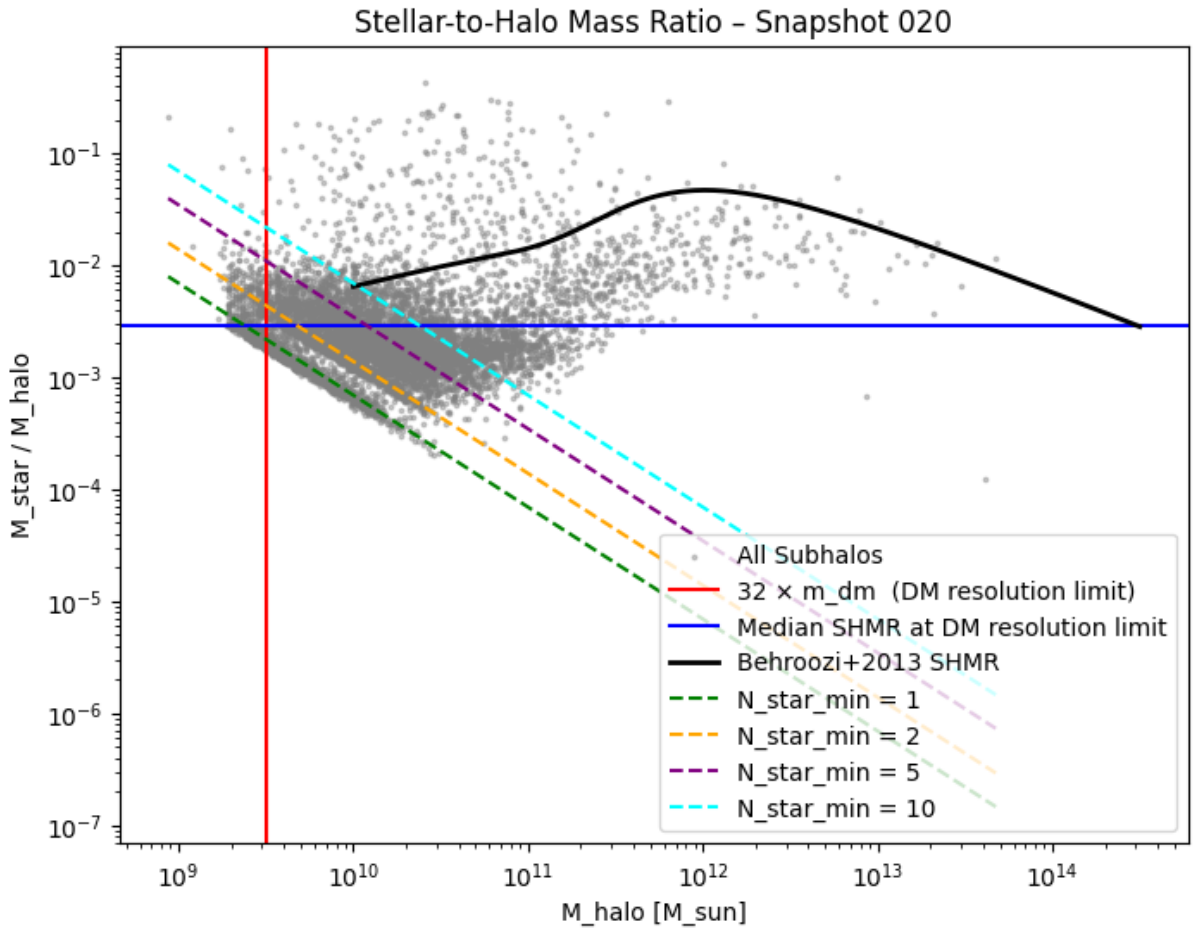

Figure 3: Stellar-to-Halo Mass Ratio for the largest identified subhalo in snapshot 20 of the L25N256 simulation, containing 72072 star particles. The plot includes the *Behroozi2013* curve alongside different resolution limits for comparison.

6 Grid Generation

6.1 Overview

The grid-generation stage constructs the precomputed stellar population synthesis (SPS) grids required for efficient spectral energy distribution (SED) interpolation, utilizing the internal grid functions of the `FSPS` Python package. Instead of evaluating `FSPS` individually for every stellar particle during runtime, spectra are precomputed once on a regular parameter grid. During SED computation, the pipeline interpolates within this grid using the respective stellar ages and metallicities. This approach significantly reduces the computational cost when processing large galaxy samples.

The core grid-generation and loading workflow is managed by the following components:

- `fsps.StellarPopulation(...)`
- `class FSPSGrid`
- `load_fsps_grid(...)`

6.2 Grid Parameter Space

The current implementation generates spectral grids as a function of:

- stellar age,
- stellar metallicity,
- wavelength.

The generated spectra additionally depend on the selected:

- stellar evolution tracks,
- spectral libraries,
- initial mass function (IMF),
- wavelength sampling configuration.

The modular structure of the pipeline allows alternative SPS models or custom parameter configurations to be integrated with minimal modifications.

The default configuration provided by `FSPS` includes MIST tracks, the MILES spectral library, and a Chabrier IMF. For alternative libraries or to include complex physics such as dust attenuation, please refer to the official documentation (see Chapter 2.3).

The baseline code provides a simple SSP (Simple Stellar Population) grid using the MIST/MILES/Chabrier combination in logarithmic space, omitting nebular emission and dust considerations. The following grid parameters should be adapted to match the target dataset:

```
# Grid resolution
N_AGE = 100                # Number of grid points for stellar age
N_LOGZ = 40                # Number of grid points for metallicity

# Grid boundaries (age in Gyr, metallicity in log10(Z/Z_sun))
AGE_MIN_GYR = 1e-3
AGE_MAX_GYR = 13.8
LOGZ_MIN = -2.0
LOGZ_MAX = 0.5
```

Note that the metallicity parameters `LOGZ_MIN` and `LOGZ_MAX` do not represent absolute physical values, but are defined relative to the solar metallicity, $Z_\odot$, in logarithmic space:

$$\texttt{logzsol} = \log_{10}\left(\frac{Z}{Z_\odot}\right). \tag{1}$$

Consequently, a value of `LOGZ_MIN` $= -2.0$ corresponds to a sub-solar metallicity of $10^{-2}\,Z_\odot$ (i.e., 1% of the solar value), while `LOGZ_MAX` $= 0.5$ represents a super-solar metallicity of $10^{0.5}\,Z_\odot$ (approximately $3.16\,Z_\odot$). Additionally, make sure to use the correct $Z_{\rm SUN}$ value for each stellar evolution track ($\text{Z\_SUN}_{\rm MIST} = 0.014$ and $\text{Z\_SUN}_{\rm Padova} = 0.01524$).

6.3 Grid Storage Structure

Generated `FSPS` grids are stored as HDF5 files. The internal structure of the resulting file is specified in Table 3. The output file is saved within the `BigData/FSPS_Grids/` directory, following the naming convention defined by the absolute path variable:

```
# Target output path for the generated grid file
OUTFILE =   "/home/<username>/Synthesizer_Osaka/BigData/FSPS_Grids/
            fsps_parsec_basel_chabrier.h5"

# Adapt the name: fsps_parsec_basel_charbrier for your girds.
```

Table 3: Expected FSPS grid structure.

Dataset	Shape	Description
`ages_gyr`	$(N_{\rm age})$	Stellar ages in Gyr
`logzsol`	(N_Z)	$\log_{10}(Z/Z_\odot)$ values
`wavelength`	(N_λ)	Wavelength grid in Å
`spectra`	$(N_{\rm age}, N_Z, N_\lambda)$	Precomputed spectral grid
`attrs["Z_sun"]`	scalar	Solar metallicity reference value

6.4 Computational Notes

Grid generation is computationally expensive but typically only needs to be executed once per SPS configuration. The runtime and storage requirements scale primarily with:

- wavelength resolution,
- age-grid density,

- metallicity-grid density.

Excessively high-resolution grids can grow to several gigabytes in size, which is not recommended as it severely degrades the performance of the subsequent SED interpolation.

7 SED Computation

7.1 Overview

The SED-computation stage forms the scientific core of the pipeline. For each stellar particle, the pipeline computes stellar ages and formation redshifts, interpolates the corresponding FSPS spectrum from the precomputed SPS grids, mass-weights the resulting spectra, and sums them to obtain integrated galaxy spectral energy distributions (SEDs).

The primary computation is performed by:

```
compute_galaxy_sed(...)
```

which handles interpolation, memory management, and spectrum accumulation. The full pipeline runs automatically by calling:

```
    master_compute_seds(...)
```

The exact algorithm sequence for the spectral energy distribution generation is schematized in Figure 4.

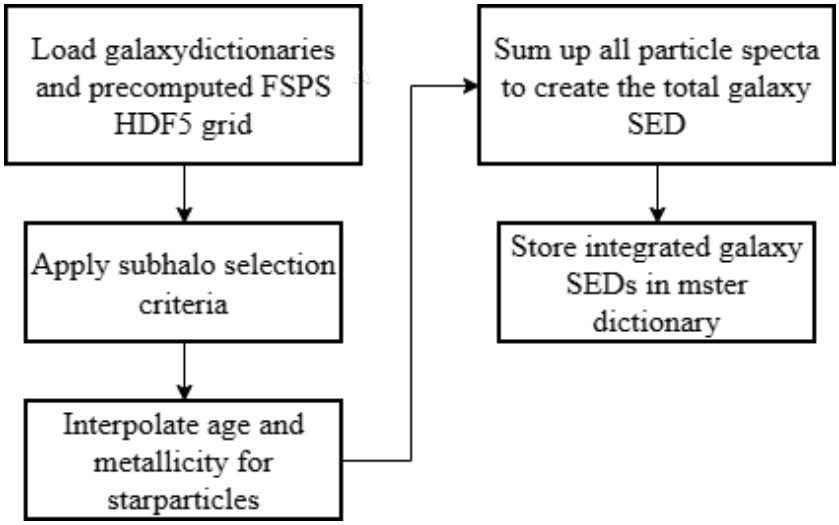

Figure 4: Sequential workflow of the SED computing stage, from subhalo selection to the integrated galaxy spectrum generation.

7.2 Stellar Age and Redshift Computation

GADGET-4 stores stellar formation times as scale factors a_{form} rather than physical ages. The formation redshift is computed via:

$$z_{\text{form}} = \frac{1}{a_{\text{form}}} - 1.$$

Similarly, the snapshot redshift is obtained from the snapshot scale factor:

$$z_{\text{snap}} = \frac{1}{a_{\text{snap}}} - 1.$$

The stellar age is then computed as:

$$t_{\text{age}} = t(z_{\text{snap}}) - t(z_{\text{form}}),$$

where $t(z)$ denotes the cosmological age of the Universe at redshift z. The cosmological calculations are performed using `astropy.cosmology.FlatLambdaCDM` with cosmological parameters read directly from the snapshot headers and are handled by:

- `compute_star_ages(...)`,
- `compute_redshifts(...)`

7.3 Interpolation Strategy

SED interpolation is performed in logarithmic age–metallicity space:

$$\log_{10}(\text{Age}) - \log_{10}(Z/Z_{\odot}).$$

The spectra themselves are additionally interpolated logarithmically in order to improve numerical stability across the large dynamic luminosity range of stellar populations. For each stellar particle, the code:

1. clips stellar ages to the grid range,
2. converts metallicities to $\log_{10}(Z/Z_{\odot})$,
3. clips metallicities to the grid boundaries,
4. identifies the surrounding grid cell,
5. performs bilinear interpolation in log-space,
6. exponentiates the interpolated spectra.

Values outside the FSPS grid are clipped to the nearest available grid boundary.

7.4 Execution Modes

The core function `compute_galaxy_sed()` supports multiple execution modes optimized for different hardware environments, as summarized in Table 4.

Table 4: SED computation modes.

Mode	Intended Usage	Memory Behavior
`laptop`	Large galaxies on limited RAM	Grouped interpolation
`hpc`	High-memory systems	Vectorized interpolation
`auto`	Automatic mode selection	Adaptive

`laptop`: Groups particles by neighboring FSPS grid cells to avoid allocating large $N_{\text{particles}} \times N_\lambda$ arrays simultaneously. While slower, this mode is highly memory efficient. Note that it can introduce minor offsets in the resulting SEDs compared to the `hpc` mode. It is directly built into `compute_galaxy_sed()`.

`hpc`: Performs highly vectorized interpolation for a large number of particles simultaneously via `interpolate_sed()`. This mode maximizes computational speed but requires substantial RAM. To mitigate memory bottlenecks, extremely large galaxies can optionally be processed in chunks.

`auto`: Automatically switches between the `laptop` and `hpc` strategies depending on the total number of stellar particles, guided by the `chunk_size` parameter.

7.5 Call the Main Function

In order to run the complete pipeline, the master wrapper function must be called as follows:

```
master_compute_seds(
    gal_root=gal_root,
    snap_list=[20],          # List of snapshot numbers to process, e.g., [10, 15, 20]
    grids=fsps_grids,
    mode="laptop",
    max_galaxies=None        # Set to an integer (e.g., 10) for a test run
)
```

The `snap_list` parameter specifies the snapshot numbers to be processed, while `max_galaxies` can be utilized to randomly subset and compute a limited number of galaxies. Furthermore, the alternative function `test_compute_seds()` can be used to test the pipeline or verify the SED output without saving any galaxy data to the disk. Please also note that the individual SEDs for each star particle are not stored due to storage limitations. If required, this behavior can be adapted in the source code.

7.6 Scientific Assumptions

The spectral energy distributions (SEDs) currently generated by the pipeline are subject to the following physical assumptions and limitations:

- **Intrinsic and Rest-Frame:** The computed spectra represent the intrinsic stellar emission in the rest-frame of the galaxy, without cosmological redshift or Doppler broadening applied.
- **No Dust Attenuation:** No dust absorption or scattering models are currently applied; the effects of the interstellar medium (ISM) on the escaping radiation are omitted.
- **No Nebular Emission:** The SEDs reflect purely stellar continuum and line radiation, excluding any contribution from ionized gas (e.g., H II regions or nebular emission lines).

- **SPS and IMF Dependence:** The results are strictly tied to the chosen FSPS model grid, the underlying stellar evolution tracks (e.g., MIST), and the adopted initial mass function (e.g., Chabrier).
- **Simple Stellar Populations (SSPs):** Each individual stellar particle is treated as a coeval, chemically homogeneous single stellar population that co-evaluates over time.

7.7 SED Diagnostics and Visualization

To visualize the generated SEDs, the pipeline provides specialized plotting utilities. Before utilizing the diagnostic tools, ensure that the required `FSPS` grids are loaded into your environment:

```
# Load all available spectral grids from the storage directory
grid_path = "/home/<username>/Synthesizer_Osaka/BigData/FSPS_Grids/"
fsps_grids = load_fsps_grids(grid_path)
```

The primary high-level function for analyzing individual galaxies is `plot_sed_for_subhalo()`. This wrapper automatically handles the file I/O, retrieves the correct subhalo data by snapshot and index, and visualizes the spectrum.

```
# Define the specific model-variant pairs you wish to compare
# Only change the grid "name" to change them
comparison_pairs = [
    ("FSPS", "fsps_parsec_basel_chabrier"),
    ("FSPS", "...")]

# Plot the SED for a specific subhalo
plot_sed_for_subhalo(
    snapnum=20,
    idx=0,                          # 6-digit rank index of the galaxy
    pairs=comparison_pairs,
    show_lambda_L_lambda=True,      # Plot \lambda \cdot L_\lambda [erg/s] if True
    normalize=False                 # If True, normalizes to peak for shape comparison
    )
```

Key Features of the Plotting Pipeline

- **Flexible Axes Selection:** Setting `show_lambda_L_lambda=True` multiplies the spectrum by its wavelength to plot λL_λ in units of $\mathrm{erg\,s^{-1}}$. Setting it to `False` displays the raw luminosity density L_λ in $\mathrm{erg\,s^{-1}\,\AA^{-1}}$.
- **Shape Comparison via Normalization:** The `normalize=True` flag scales each spectrum to its individual peak value. This is highly useful for comparing spectral shapes across different initial mass functions (IMFs) independently of the total stellar mass.

- **Internal Fallback Controls:** Lower-level functions such as `plot_sed_pairs()` and `plot_all_seds()` are utilized internally to automatically filter and skip missing model variants without crashing the runtime loop.

A typical output generated by this diagnostic tool is illustrated in Figure 5, which compares different spectral population synthesis configurations for the same object.

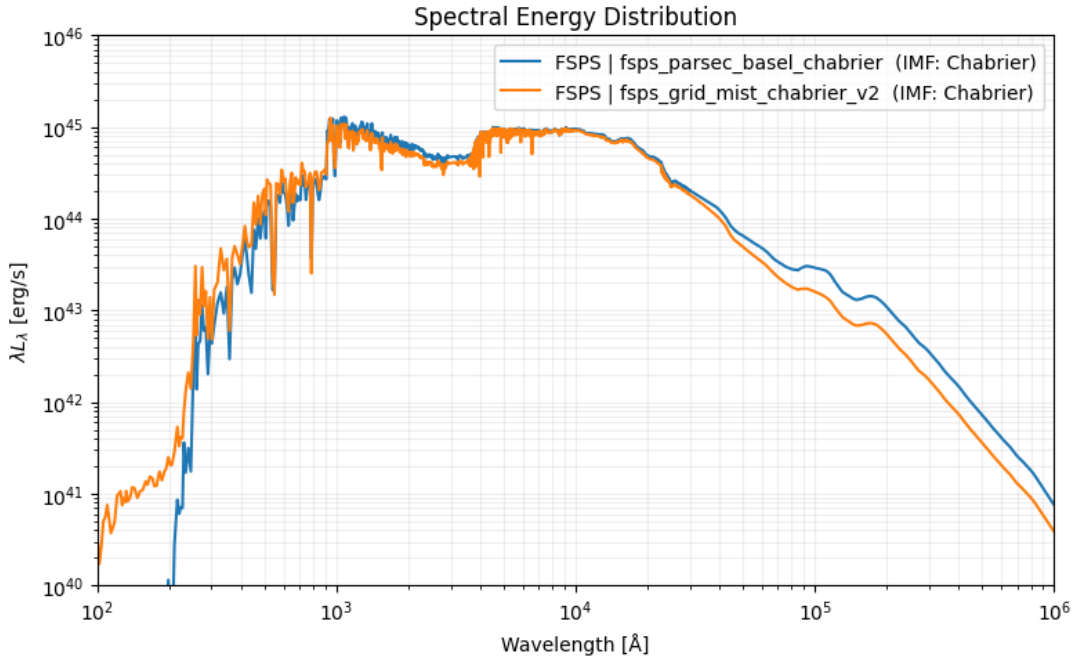

Figure 5: Spectral Energy Distribution (SED) for the largest identified subhalo in snapshot 20 of the L25N256 simulation. The plot compares the computed SEDs using the PARSEC/BaSeL/Chabrier and MIST/MILES/Chabrier combinations.

8 Photometric Synthesis

8.1 Overview

The photometric-synthesis stage convolved the generated stellar spectra with broadband filter transmission curves to derive absolute magnitudes. While the pipeline is modular and supports any filter set available via `fsps.list_filters()`, the default implementation focuses on the SDSS core bands:

`sdss_u, sdss_g, sdss_r, sdss_i, sdss_z.`

The primary execution and numerical integration loops are managed by the following components:

- `compute_galaxy_photometry()`, which coordinates the chunked particle loops and handles dictionary state storage,
- `compute_magnitude()`, which executes the core bandpass integration.

The final derivation can be called via `master_compute_photometry`. The complete architectural data flow of the spectral and photometric processing modules—highlighting the memory-saving chunking loops and the interaction with the precomputed `FSPS` grids—is schematized in Figure 6.

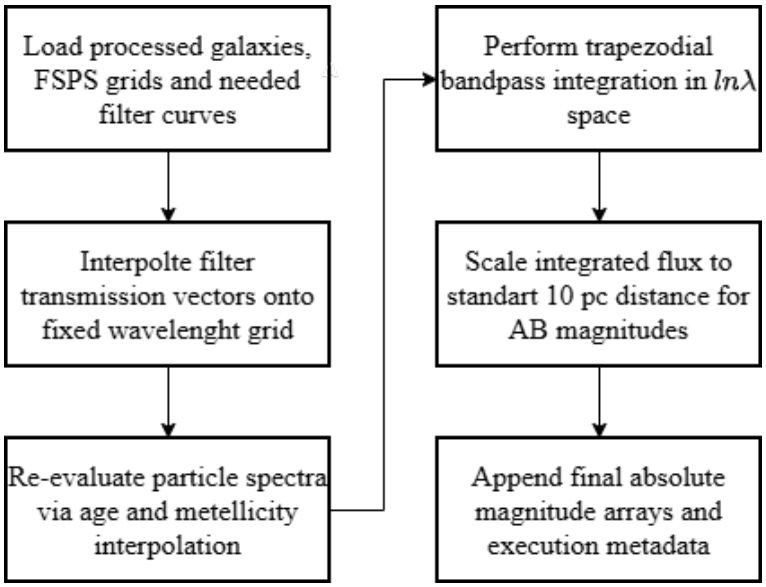

Figure 6: Sequential workflow of the photometric synthesis stage, highlighting the memory-optimized chunking mechanism and magnitude conversion.

8.2 Filter Handling and Calibration

Filter transmission curves $T(\lambda)$ are loaded via the `python-fsps` API and interpolated onto the high-resolution wavelength grid of the active SED. For each bandpass, the transmission-weighted effective wavelength λ_{eff} is computed as:

$$\lambda_{\text{eff}} = \frac{\int \lambda\, T(\lambda)\, \mathrm{d}\lambda}{\int T(\lambda)\, \mathrm{d}\lambda}. \tag{2}$$

Following the `FSPS` reference specifications for UV, optical, and near-IR photometry, the spectral slope parameter is set to $\beta = 0$. This ensures a standard photon-counting calibration where the convolved flux density perfectly matches the physical flux at the central frequency of the respective band.

8.3 Magnitude Computation and Bandpass Integration

Unlike simplified approximations, that scale a single band-averaged luminosity by the effective wavelength, the pipeline performs a mathematically rigorous numerical integration in logarithmic wavelength space ($\ln \lambda$).

For each individual stellar particle, the mass-scaled monochromatic luminosity density, L_λ (in $\mathrm{erg\,s^{-1}\,\AA^{-1}}$), is first converted into frequency space, L_ν (in $\mathrm{erg\,s^{-1}\,Hz^{-1}}$), at every grid point:

$$L_\nu(\lambda) = L_\lambda(\lambda) \cdot \frac{\lambda^2}{c}, \tag{3}$$

where c represents the speed of light. The band-averaged spectral luminosity density, $\langle L_\nu \rangle$, is subsequently evaluated using the trapezoidal rule:

$$\langle L_\nu \rangle = \frac{\int L_\nu(\lambda)\, T(\lambda)\, \mathrm{d}\ln\lambda}{\int T(\lambda)\, \mathrm{d}\ln\lambda}. \tag{4}$$

To determine the absolute AB magnitude, the mean luminosity density is converted to a physical flux density, F_ν, at a fixed standard cosmological reference distance of $d_{10\mathrm{pc}} = 10\,\mathrm{pc}$ ($3.0857 \times 10^{19}\,\mathrm{cm}$):

$$F_\nu = \frac{\langle L_\nu \rangle}{4\pi d_{10\mathrm{pc}}^2}. \tag{5}$$

Finally, the absolute AB magnitude m_{AB} is derived using the standard zero-point:

$$m_{\mathrm{AB}} = -2.5 \log_{10}(F_\nu) - 48.60. \tag{6}$$

8.4 Scientific Assumptions and Boundary Conditions

The synthesized photometric quantities represent idealized physical outputs and are subject to the following characteristics:

- **Intrinsic and Rest-Frame:** Magnitudes reflect the unattenuated stellar emissions in the rest-frame of the galaxy, without accounting for cosmological redshift ($z = 0$) or line-of-sight velocity broadening.
- **No Interstellar Medium (ISM) Effects:** No dust attenuation (absorption or scattering) or nebular emission lines from ionized gas regions are included.
- **Resolution Limits:** Calculations are performed per stellar particle acting as a coeval Simple Stellar Population (SSP), independent of instrument-specific observational noise or sky background configurations.

8.5 Photometric Analysis and Diagnostics

To evaluate the synthesized photometric data, the pipeline includes diagnostic utilities to generate Color-Magnitude Diagrams (CMDs) and track stellar populations. A typical scientific output is illustrated in Figure 7, showing the resolved stellar population of a single massive system.

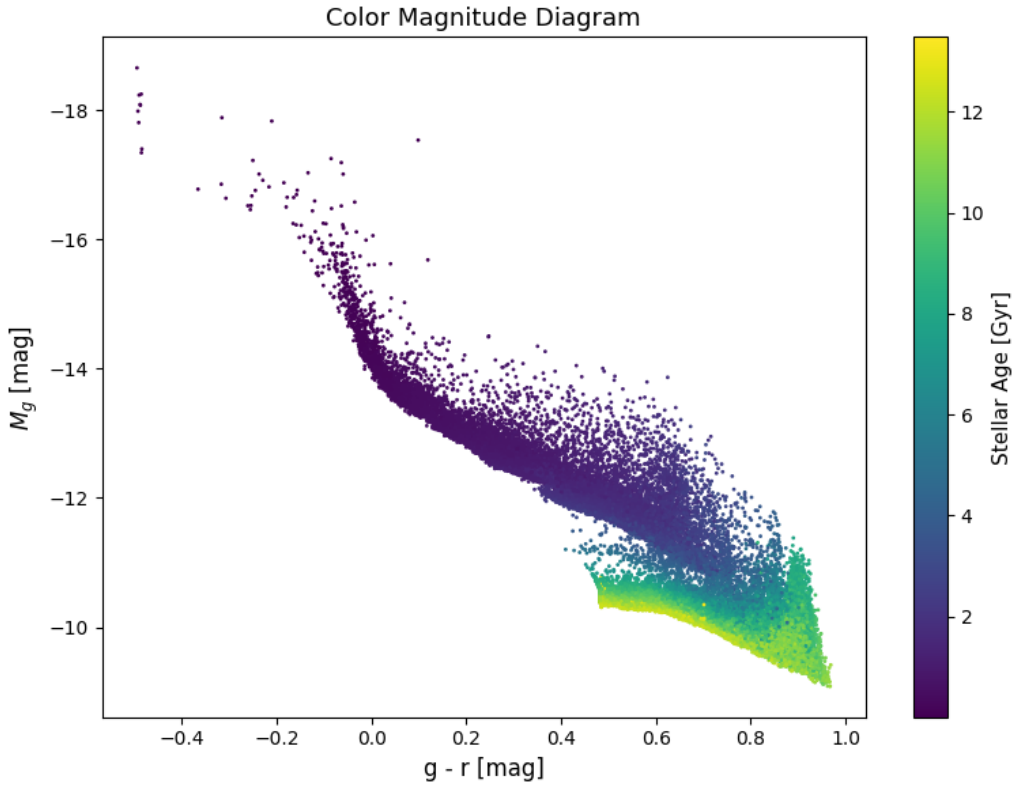

Figure 7: Color-Magnitude Diagram ($g - r$ vs. M_r) for all individual stellar particles within the largest identified subhalo in snapshot 20 of the L25N256 simulation.

9 Analysis and Visualization

The analysis and visualization snippets provides a comprehensive set of diagnostic and exploratory tools designed to validate both intermediate pipeline states and final data products. While these utilities are decoupled from the core SED generation and photometric synthesis execution loops, they are critical for verifying data quality, exploring physical galaxy samples, and diagnosing potential systematic processing anomalies.

The diagnostic toolkit encompasses a variety of scientific plotting functions, including:

- **Structural and Environmental Diagnostics:** Halo mass histograms, stellar particle count distributions, and subhalo spatial distributions.
- **Scaling Relations and Photometry:** The Stellar-to-Halo Mass Relation (SHMR, see Figure 3), multi-model SED comparisons (see Figure 5), resolved Color-Magnitude Diagrams (CMDs, see Figure 7), and age–color relations.

The architecture of these analysis utilities is intentionally modular. This separation ensures that the core pipeline remains lightweight, while providing an easily extensible framework for implementing custom diagnostic plots or expanding into additional scientific applications.

10 Performance and Scaling

10.1 Runtime Scaling

The execution time of the pipeline is determined by a combination of physical sample sizes, grid configurations, and hardware limitations. The primary scaling factors include:

- the total number of snapshots and selected subhalos,
- the stellar particle density within individual galaxies,
- the number of concurrent `FSPS` grid variants and their respective wavelength resolution (N_λ),
- the input/output (I/O) throughput of the underlying filesystem during data loading and storage.

Assuming ideal I/O conditions, the computational complexity of the core SED interpolation scales linearly with the total number of processed particles, the spectral resolution, and the number of grid configurations:

$$\mathcal{O}\left(N_{\text{particles}} \times N_\lambda \times N_{\text{grids}}\right). \tag{7}$$

10.2 Memory Scaling

The most memory-intensive operation in spectral synthesis is the handling of unreduced, per-particle spectra. Allocating a full multidimensional array for a large system scales as:

$$\mathcal{O}\left(N_{\text{particles}} \times N_\lambda\right). \tag{8}$$

For massive galaxies, this allocation quickly becomes prohibitive for standard computing environments. For instance, a well-resolved subhalo containing 10^6 stellar particles evaluated on a grid with 10^4 wavelength bins requires an array of 10^{10} floating-point elements. In double-precision format (`float64`), this single matrix occupies approximately 80 GB of memory, which exceeds the physical RAM of standard workstations.

To mitigate these memory bottlenecks, the pipeline implements dedicated memory management strategies that can be toggled via the execution modes:

- **Stellar Particle Chunking:** Subdivides massive arrays into smaller, cache-friendly blocks defined by the `chunk_size` parameter.
- **Grouped Interpolation:** Clusters particles by neighboring grid cells to reduce redundant interpolation steps.
- **Laptop vs. HPC Optimization:** Provides an adaptive memory layout, allowing low-RAM systems to prioritize sequential grouped execution over full vectorization.

10.3 Storage Scaling

The permanent storage footprint on disk scales as additional post-processing layers (SEDs and photometry arrays) are appended to the master galaxy dictionaries. The total disk space requirements are driven by:

- raw and derived particle property arrays,
- integrated, high-resolution galaxy SED spectra,
- synthesized broadband photometry arrays and associated metadata blocks.

The storage footprint of the synthesized broadband photometry scales strictly with the number of targeted filter bands:

$$\mathcal{O}\left(N_{\text{particles}} \times N_{\text{filters}}\right). \tag{9}$$

Crucially, because the high-resolution spectra are integrated into a single, global galaxy spectrum during the initial phase, the storage requirement for the final SED data remains negligible compared to the raw particle catalogs.

10.4 Computational Redundancy and Memory Optimization

As detailed in Section 3, individual stellar particle SEDs are directly summed during the primary SED computation phase to prevent prohibitive storage requirements. Consequently, `compute_galaxy_photometry()` temporarily re-calculates the monochromatic SEDs for all star particles on-the-fly using `interpolate_sed()`.

To prevent memory (RAM) overflows during this redundant calculation, the particle array is processed in sub-volumes defined by `chunk_size`. While storing these individual particle SEDs permanently is possible by overriding the pipeline's default saving configurations, re-evaluating them during the photometric synthesis stage represents a highly optimized trade-off that prioritizes low storage footprints over minor runtime overheads.

11 Known Limitations and Future Development

11.1 Numerical and Algorithmic Assumptions

The numerical framework of the pipeline relies on specific grid boundaries and interpolation techniques that introduce the following constraints:

- **Grid Boundary Clipping:** Stellar particle ages and metallicity, that fall outside the precomputed `FSPS` parameter space are automatically clipped to the minimum or maximum grid boundaries. This can cause artificial saturation for extremely young, old, or metal-poor/metal-rich populations.
- **Bilinear Grid Interpolation:** The mapping of particle properties onto the spectral grid is performed via bilinear interpolation within the $\log_{10}(\text{age})$ and $\log_{10}(Z/Z_{\odot})$ parameter space, assuming local linearity between grid points.
- **Logarithmic Spectral Scaling:** The individual monochromatic flux densities are interpolated in logarithmic space to maintain numerical stability across several orders of magnitude in spectral intensity.

11.2 Implementation Constraints

Certain software architectural shortcuts have been adopted to optimize the pipeline for immediate performance, resulting in the following system limitations:

- **Single-Chunk Particle Extraction:** The current particle extraction algorithm implicitly assumes that all stellar particles belonging to a given subhalo are contained within a single snapshot file chunk. Cross-chunk boundary populations are not yet handled.
- **Disabled I/O Backups:** To maximize filesyste writing speeds and prevent storage overheads, automated backup mechanisms for overwriting existing galaxy files are disabled by default.
- **Interactive Analysis Dependencies:** Several diagnostic and plotting utilities are designed for interactive execution (e.g., within Jupyter Notebooks) and assume that the required data structures have been pre-allocated in the active runtime environment.

11.3 Current Physical Simplifications

To maintain a lightweight baseline configuration, the pipeline omits complex secondary astrophysics and observational effects. The generated data products currently do not incorporate:

- **Interstellar Medium (ISM) Physics:** Dust attenuation (absorption and scattering), nebular emission lines from ionized gas regions, and active radiative transfer.
- **Observational and Instrument Effects:** Cosmological redshifting of spectra to the observer frame ($z > 0$), telescope or detector throughput curves, finite aperture constraints, and realistic observational noise configurations.

11.4 Future Development

The modular design of the pipeline allows for seamless scaling. Future development cycles are targeted toward resolving the current constraints through the following upgrades:

- **Performance and Scalability:** Implementation of chunk-safe particle extraction routines alongside distributed parallel processing architectures (`multiprocessing` or MPI) to accelerate large-volume cosmological simulation processing.
- **Advanced Astrophysics Modules:** Integration of standard empirical dust attenuation curves (e.g., Calzetti or Charlot & Fall) and `FSPS`-native nebular emission models to synthesize more realistic spectra.
- **Observational Mock Pipelines:** Development of an observer-frame mock photometry module, including a wider selection of international survey filter sets, instrumental responses, and automated aperture measurements.
- **Software Quality Assurance:** Deployment of automated validation test suites and transitioning the execution loop to a configuration-file-driven system (e.g., YAML or TOML) for enhanced reproducibility.

12 Function Reference

This section lists the main functions of the pipeline. Each function is grouped by pipeline block.

12.1 Stage 1: Snapshot Processing Reference

This block manages the automated discovery, file I/O operations, structural merging, and physical extraction of subhalo and stellar particle catalogs. The individual core functions are specified below:

`detect_all_snapnums(basepath)`
Scans the simulation directory to identify available snapshot outputs.

- **Parameters:** `basepath` (str) – Path to the raw simulation data.
- **Returns:** A sorted, unique list of detected snapshot numbers.

`load_all_groups_for_snapnum(basepath, snapnum)`
Opens all distributed Friends-of-Friends (FoF) and subhalo catalog files.

- **Parameters:** `basepath` (str) – Path to data; `snapnum` (int) – Snapshot number.
- **Returns:** A list of open read-only `h5py.File` objects for the group catalogs.

`load_all_snapshots_for_snapnum(basepath, snapnum)`
Opens all distributed raw particle snapshot files for a given time step.

- **Parameters:** `basepath` (str) – Path to data; `snapnum` (int) – Snapshot number.
- **Returns:** A list of open read-only `h5py.File` objects containing particle blocks.

`merge_group_subhalos(group_files)`
Concatenates subhalo properties from fragmented group catalog files into contiguous arrays.

- **Parameters:** `group_files` (list) – List of open group file handles.
- **Returns:** A tuple containing total subhalo masses (`M_halo`), particle counts per type (`N_type`), global indexing offsets (`Off_type`), and comoving 3D spatial positions (`pos`).

`merge_group_fof(group_files)`
Reads and merges macro-group attributes across all available catalog chunks.

- **Parameters:** `group_files` (list) – List of open group file handles.
- **Returns:** A tuple containing the index of the first subhalo per group (`GroupFirstSub`) and the total number of subhalos bound to each FoF group (`GroupNsubs`).

`build_subhalo_to_fof(GroupFirstSub, GroupNsubs, N_subhalos)`
Constructs an inverse lookup array mapping each individual subhalo to its parent FoF group.

- **Returns:** An integer array of length N_{subhalos} storing parent FoF IDs. Isolated objects without an FoF membership are flagged with `-1`.

`build_star_index(snap_files)`
Computes cumulative stellar particle offsets across distributed snapshot files.

- **Returns:** A tuple storing the raw particle counts per sub-file and the cumulative prefix-sum offsets used for global-to-local coordinate translation.

`get_hubble_param(snap_files)`
Reads the dimensionless Hubble parameter h from the snapshot header for unit conversions.

- **Returns:** Floating-point scalar h.

`extract_star_particles_global(...)`
Extracts physical properties for a specific block of star particles using a global offset pointer.

- **Returns:** A dictionary containing calibrated particle data: masses ($M_\odot$), comoving coordinates, formation scale factors a, and total metallicity.
- **Limitation:** Assumes that the selected target population does not cross distributed file chunk boundaries.

`process_galaxies_in_snapshot(...)`
Extracts, filters, sorts by descending stellar mass, and serializes all eligible subhalo dictionaries for a single snapshot into separate binary `.npy` files.

- **Selection Logic:** Evaluates physical resolution thresholds via a localized Stellar-to-Halo Mass Relation (SHMR) envelope to calculate $N_{\text{star,min}}$.

`process_all_snapshots(basepath, save_galaxies)`
Acts as the master pipeline orchestration wrapper. It sequences the entire workflow from automated data discovery and catalog merging up to mass calculations and final file serialization.

The entire sequence for a simulation directory can be executed via the master wrapper utilizing the syntax shown in Listing 12.1:

```
process_all_snapshots(
    basepath="../../BigData/",
    save_galaxies=True
)
```

12.2 Stage 2: Population Synthesis Grid Generation Reference

This module coordinates the precomputation of the high-resolution Simple Stellar Population (SSP) spectral grids utilizing the `python-fsps` framework. By compiling these grids across a regularized age-metallicity parameter space and caching them into compressed HDF5 files, the pipeline bypasses expensive runtime Fortran calls during the main galaxy SED evaluations.

Grid Parameter Specifications

The configuration parameters established at the head of the generation script define the resolution limits and underlying stellar physics of the synthesized grid:

- `OUTFILE` (str) – Absolute target filepath for the generated HDF5 grid matrix.
- `Z_SUN` (float) – Solar metallicity mass fraction reference (e.g., 0.014 for MIST, 0.01524 for PARSEC).
- `IMF_TYPE` (int) – Initial Mass Function selector mapped to the `FSPS` core library (set to `1` for a Chabrier IMF).
- `N_AGE` / `N_LOGZ` (int) – Grid density settings configuring the total number of evaluation nodes (default: 100×40).
- `AGE_MIN_GYR` / `AGE_MAX_GYR` (float) – Continuous boundary bounds for stellar population aging (1 Myr to 13.8 Gyr).
- `LOGZ_MIN` / `LOGZ_MAX` (float) – Metallicity boundaries defined in logarithmic solar units $\log_{10}(Z/Z_{\odot})$ ranging from -2.0 to 0.5.

Algorithmic Step-by-Step Execution

The compilation script executes sequentially through the following operations, as reflected in the pipeline architecture:

1. **Object Initialization:**
   Instantiates a single master `fsps.StellarPopulation` object. For numerical safety and continuum integrity, secondary physics variables are explicitly disabled (`add_neb_emission = False`, `dust2 = 0.0`, `fagn = 0.0`).

2. **Coordinate Space Definition:**
   Generates a logarithmic axis array for stellar ages using `np.logspace()` and a linear axis array for metallicities via `np.linspace()`.

3. **Wavelength Initialization:**
   During the first loop iteration ($j = 0$), the pipeline executes a query call to `sp.get_spectrum()` to capture the high-resolution wavelength grid. This vector length (N_{λ}) is used to dynamically structure the master 3D dataset array.

4. **HDF5 High-Performance Caching:**
   Iterates through the metallicity nodes. For each node, a temporary 2D block matrix ($N_{\text{age}} \times N_{\lambda}$) is populated by scaling the raw spectra by the solar luminosity constant ($L_{\odot} = 3.828 \times 10^{33}\,\text{erg}\,\text{s}^{-1}$). The slice is then streamed into the HDF5 tensor via compressed chunked I/O allocation (`chunks = (1, 1, n_lambda)` using `gzip` compression).

HDF5 Metadata Attribute Mapping

Upon completion of the grid synthesis loop, the pipeline attaches a permanent metadata block directly to the file attributes (`f.attrs`) to guarantee full reproducibility. These tracks store the exact configuration state:

Table 5: Cached HDF5 grid metadata attributes.

Attribute Key	Data Type	Description
`Z_sun`	float	Mass fraction of solar metallicity reference
`imf_type`	int	Code index mapping the chosen IMF configuration
`zcontinuous`	int	FSPS interpolation flag (enforced to 1)
`units`	str	Physical unit tracking string (`erg/s/Angstrom per Msun`)
`fsps_version`	str	Version string tag of the active `python-fsps` instance
`build_time_seconds`	float	CPU execution wall time tracking metric

12.3 Stage 3: SED Computation Reference

This processing block controls the creation of stellar population grids, cosmological runtime conversions, log-flux billing grid interpolations, and the multi-snapshot master loop execution. The core functions and containers are detailed below:

`class FSPSGrid(path, imf=None)`

An object-oriented container handling the ingestion and structural management of a precomputed `FSPS` spectral grid matrix stored in HDF5 format.

- **Attributes Loaded:** Stellar age nodes (`ages`, Gyr), logarithmic metallicity nodes (`logz`), wavelength bins (`wave`, Å), the 3D specific luminosity tensor (`spec`, $\mathrm{L}_\odot\,\text{Å}^{-1}\,\mathrm{M}_\odot^{-1}$), and the solar metallicity reference fraction (`Z_sun`).
- **Internal Optimizations:** Pre-allocates log-axes and transforms the 3D tensor into logarithmic space ($\mathbf{S}_{\log} = \log_{10}(\mathbf{S} + 10^{-30})$) to avoid numerical floating-point underflows. It also includes the heuristic path parser `_infer_imf_from_filename()` to automatically identify the Initial Mass Function type.

`load_fsps_grids(grid_dir)`

An I/O utility that scans the targeted folder for serialized `.h5` grid files, instantiates them as individual `FSPSGrid` objects, and returns them packed within a nested lookup configuration dictionary.

- **Returns:** A nested lookup dictionary tracking the data variants: `{"FSPS": {variant_name: FSPSGrid}}`.

`compute_star_ages(gal, params, header)`

Translates raw GADGET particle formation scale factors (a_{form}) into physical lookback ages expressed in Gigayears (Gyr).

- **Methodology:** Invokes an Astropy `FlatLambdaCDM` cosmology class parameterized by the header-derived values for H_0 and Ω_0. It converts scale factors to cosmic lookback times to isolate the differential age coordinate for each particle via:

$$t_\star = t_{\text{cosmo}}(z_{\text{snap}}) - t_{\text{cosmo}}(z_{\text{form}}). \tag{10}$$

`compute_redshifts(gal, header)`
Derives standard cosmological redshifts from scale factors in-place.

- **Returns:** The updated galaxy data dictionary now hosting explicit array mappings for both the global system rest-horizon (`"z_snap"`, scalar) and individual stellar particles (`"z_form"`, 1D array).

`interpolate_sed(grid, ages, Z)`
Executes a fully vectorized bilinear interpolation inside log-age and log-metallicity parameter space for N star particles simultaneously.

- **Returns:** A 2D array of shape $(N_{\text{particles}} \times N_\lambda)$ storing the monochromatic specific luminosity profiles.
- **Numerical Control:** Bracketing index nodes are isolated using binary search trees (`np.searchsorted()`) and clamped to grid boundaries. Spectral combinations are combined inside log-flux space to guarantee dynamic continuum path stability before exponential reconstruction.

`compute_galaxy_sed(gal, grid, mode, chunk_size)`
Compiles the total integrated spectral energy distribution of a subhalo by summing the mass-weighted continuums of its constituent stellar particle elements.

- **Memory Configurations:** If running in `laptop` mode, the pipeline aggregates particles into grid cells via `np.unique()`, eliminating the need to allocate heavy intermediary $N \times N_\lambda$ matrix fields in memory. In `hpc` mode, full vectorization is applied, with high-RAM protection controlled through uniform slicing boundaries specified by the `chunk_size` parameter.

`master_compute_seds(gal_root, snap_list, grids, mode, ...)`
Acts as the master processing orchestration wrapper across multiple simulation outputs. It automates cosmological initialization, loops through snapshot subfolders, identifies incomplete or corrupted dataset keys, applies grid interpolation routines, and serializes the state outputs directly back into the target galaxy files.

`test_compute_seds(...)`
A lightweight, decoupled diagnostic function used to execute sample grid interpolation tests and verify spectral continuum calculations without writing any galaxy data streams to disk.

The full synthesis routine across multiple snapshots can be orchestrated via the master execution wrapper using the standard syntax detailed in Listing 12.3:

```
# Load existing models from the HDF5 repository directory
grid_dir = "/home/<username>/Synthesizer_Osaka/BigData/FSPS_Grids/"
fsps_grids = load_fsps_grids(grid_dir)

# Establish target storage data directory path
gal_root = "/home/<username>/Synthesizer_Osaka/DataSave/Galaxies.
    Storage"

# Execute integrated SED computation loop in memory-safe laptop
    mode
```

```
master_compute_seds(
    gal_root=gal_root,
    snap_list=[20],
    grids=fsps_grids,
    mode="laptop"
)
```

12.4 Block 4: Photometry Reference

This processing block controls the loading and grid-interpolation of broadband filter transmission curves, the high-precision numerical bandpass integration, and the multi-snapshot master loop execution for photometric synthesis. The core unctions are detailed below:

`load_sdss_filters(wave_sed)`
Loads the requested SDSS filters from the core library and interpolates their relative transmission profiles onto the active SED wavelength grid.

- **Parameters:** `wave_sed` (1D array) – The master wavelength axis of the active grid (Å).
- **Returns:** `filters` (dict) – A dictionary containing the interpolated transmission curves (`"trans_interp"`, zeroed outside filter boundaries) and the transmission-weighted effective wavelengths (`"lambda_eff"`).
- **Default Bands:** Automatically maps the standard optical coverage: `sdss_u`, `sdss_g`, `sdss_r`, `sdss_i`, `sdss_z`.

`compute_magnitude(sed_lambda, mass, wave, trans)`
Computes the absolute AB magnitude for a single stellar particle through a designated filter bandpass.

- **Mathematical Logic:** Converts the mass-scaled monochromatic luminosity density $L_\lambda = L_{\lambda,\mathrm{SSP}} \times m_\star$ ($\mathrm{erg\,s^{-1}\,\AA^{-1}}$) into frequency space $L_\nu = L_\lambda \lambda^2/c$. It solves for the band-averaged flux profile $\langle F_\nu \rangle$ at a fixed cosmological standard reference distance of $d_{10\mathrm{pc}} = 10\,\mathrm{pc}$ ($3.0857 \times 10^{19}\,\mathrm{cm}$) using numerical trapezoidal integration over logarithmic wavelength space ($\ln\lambda$):

$$\langle F_\nu \rangle = \frac{1}{4\pi d_{10\mathrm{pc}}^2} \frac{\int L_\nu(\lambda)\, T(\lambda)\, \mathrm{d}\ln\lambda}{\int T(\lambda)\, \mathrm{d}\ln\lambda}. \tag{11}$$

The absolute AB magnitude is then derived using the standard zero-point:

$$M_{\mathrm{AB}} = -2.5 \log_{10}\langle F_\nu \rangle - 48.60. \tag{12}$$

If the integrated flux is non-positive, the function handles the exception by returning `np.nan`.

`compute_galaxy_photometry(gal, grids, chunk_size=10000, overwrite=False)`
Computes per-particle SDSS photometry for all active grid models and structural variants of a single galaxy.

- **Memory Mitigation:** To prevent memory overflows during the redundant on-the-fly particle spectrum reconstruction, the total particle array is processed in sub-blocks controlled through the `chunk_size` parameter. For each chunk, the particle SEDs are temporarily re-interpolated via `interpolate_sed()` and fed into the integration loops before the transient memory slices are discarded.
- **Returns:** The updated galaxy data dictionary now housing the calculated absolute magnitudes (`M_u, M_g, M_r, M_i, M_z`) and a comprehensive execution metadata block (`photometry_meta`).

`master_compute_photometry(gal_root, snap_list, grids, ...)`
Acts as the master execution wrapper for the photometry pipeline across multiple snapshots. It coordinates automatic directory routing, filters targeted files via index masks (`galaxy_ids`), handles dictionary I/O streams, and saves the updated galaxy records in-place.

The full photometric post-processing routine across the simulation repository can be orchestrated via the master loop syntax detailed in Listing 12.4:

```
# Load existing model grids into memory environment
grid_dir = "/home/<username>/Synthesizer_Osaka/BigData/FSPS_Grids/"
fsps_grids = load_fsps_grids(grid_dir)

# Establish absolute directory path to individual galaxy files
gal_root = "/home/<username>/Synthesizer_Osaka/DataSave/Galaxies.
    Storage"

# Execute automated photometry synthesis pipeline across targeted
    snapshots
master_compute_photometry(
    gal_root=gal_root,
    snap_list=[20],
    grids=fsps_grids
)
```

13 References

References

[1] Springel, V. et al. 2021, *Simulating cosmic structure formation with the GADGET-4 code*, MNRAS, 506, 2871.

[2] Conroy, C., Gunn, J. E., & White, M. 2009, *The Propagation of Uncertainties in Stellar Population Synthesis Modeling*, ApJ, 699, 486.

[3] Behroozi, P. S., Wechsler, R. H., & Conroy, C. 2013, *The Average Star Formation Histories of Galaxies in Dark Matter Halos from $z = 0$–8*, ApJ, 770, 57.